

HACKATHON

Agentic AI

Trading Agent on a Simulated Market

Two days • Teams of 3–4 people

1. Mission Statement

AI agents are not chatbots — they are systems that perceive, reason, and act autonomously. In this hackathon you will build an agent capable of operating on a simulated stock market: it will read market data and news, decide what to buy or sell while explaining its choices in natural language, execute orders via API, and learn from its own mistakes through an automatic trade journal.

The goal is not to build the most profitable agent — it is to build the most "intelligent" one: the agent that handles uncertainty best, does not hallucinate price data, stops when it does not know what to do, and explains its decisions in a traceable way.

You will have access to a simulated broker (Alpaca Paper Trading), real delayed market data, and news search APIs. Real money: zero. Learning: maximum.

Disclaimer: this is a purely educational simulation. No real money is involved, no real orders are placed on financial markets, and nothing in this hackathon constitutes financial advice or investment guidance of any kind.

2. Challenge Scope

What you must deliver

An autonomous agent that, given a starting virtual portfolio (100,000 simulated USD), operates independently for at least 5 consecutive minutes during the final demo without human intervention.

The agent must be able to:

- Autonomously retrieve prices and market data via tool calls (not from the model's memory).
- Read and summarize relevant financial news for the monitored tickers.
- Produce explicit reasoning before each order: why to buy, why to sell, why to do nothing.

- Execute orders on the simulated broker and update the portfolio state.
- Handle errors: if a tool fails, the agent must recover without getting stuck.
- Maintain a trade journal: every decision is recorded with a timestamp, ticker, rationale, and outcome.

Ambition levels

Not all teams start from the same point. The challenge is designed with three progressive levels — every team is expected to reach at least Level 1; Levels 2 and 3 are for teams that move faster or have more prior experience. The jury evaluates each team relative to the level they declared at kick-off.

- Level 1 — Core Agent (minimum bar, reachable by everyone)

The agent retrieves live prices and news via tool calls (nothing from the model's memory), produces a written rationale for each decision (buy / sell / hold), and records every decision in a structured trade journal with timestamp, ticker, reasoning, and outcome. No order execution required at this level.

- Level 2 — Executing Agent (intermediate)

Everything in Level 1, plus: the agent executes real orders on the simulated broker, updates the portfolio state after each trade, and handles at least one tool failure gracefully (e.g. retries, fallback data source, or a principled "I cannot act" response).

- Level 3 — Autonomous Agent (advanced)

Everything in Level 2, plus: the agent runs a full autonomous loop for 30+ minutes, integrates a memory or context layer to reason across multiple decision cycles, and demonstrates a non-trivial strategy or risk-management logic (e.g. position sizing, stop-loss, sector diversification).

Two fundamental rules

Rule 1: the agent must never make up prices or data. Everything it uses to make decisions must come from a documentable tool call. This is the difference between an agent and a chatbot dressed up as one.

Rule 2: the agent must run independently and should be not triggered by the human input. Humans can answer to questions of the agent but not trigger the first action.

3. Work Plan

The schedule below defines the common milestones that all teams attend together (kick-off, tech talk, checkpoints, stand-up, final demo). Between milestones, teams manage their own time and working rhythm: working from the room, stepping out, splitting tasks, or taking a break are all fine. The only rule is to be present and ready at each milestone on time.

Day 1 — Build fast, think agentic

09:00 – 09:30 — Kick-off and team formation

- Presentation of available tracks and the simulated broker
- Team formation (3–4 people) and selection of ambition level
- Starter kit handout: template repo, API keys, Alpaca documentation
- Anti-wrapper rule: the agent must make at least one autonomous decision

09:30 – 10:00 — Tech talk: anatomy of a trading agent

- Perception → reasoning → action loop: what makes it “agentic”
- Tool use: how to structure `get_price`, `search_news`, `place_order`
- Context management: how to prevent the LLM from “inventing” data
- Alpaca Paper Trading: live API demo in 15 minutes

10:00 – 13:00 — Sprint 1: working prototype

- Goal: agent that completes at least one end-to-end cycle (fetch → decide → record). Level 1 teams: a reasoned decision logged in the journal counts. Level 2+ teams: aim to execute a real order.
- Mentors available for check-ins every 30 minutes — teams choose when to ask
- Intermediate checkpoint at 12:00: each team runs a quick live test

13:00 – 14:00 — Lunch + informal demo

- Each team shows their prototype status in 2 minutes
- No judging: quick technical feedback from mentors only
- ⚡ Mini-challenge 1: “First Trade” (see section 5)

14:00 – 17:30 — Sprint 2: additional tools and memory

- Add news search, trade journal, and robust error handling
- Optional: persistent memory across decision cycles
- ⚡ Mini-challenge 2: “Best Reasoning” at 16:00 (see section 5)

17:30 – 18:00 — Daily stand-up (attendance required)

- 3 minutes per team: what works, what is broken, plan for tomorrow
- Announcement of Day 1 mini-challenge results

Day 2 — Ship it, present it

09:00 – 09:20 — Morning sync

- Final priorities, reminder on the demo format (5 + 3 minutes)
- Code freeze announced for 15:00: no new features after that

09:20 – 13:00 — Sprint 3: polish and stress test

- Focus on robustness: what happens if a tool returns an error?
- Smooth demo path: prepare a repeatable scenario for the presentation
- No new features: stability and code clarity only
- ⚡ Mini-challenge 3: “Best Recovery” at 11:00 (see section 5)

13:00 – 15:00 — Lunch & Final Debugging

- Preparation of slides/demo script (maximum 3 context slides)
- ⚡ Mini-challenge 4: “Best Journal” at 13:00 (see section 5)

15:00 – 16:30 — Final demo

- 5 minutes of live demo per team + 3 minutes of Q&A from the jury
- The agent must run autonomously during the demo: no manual input
- Presentation order drawn by lot on the morning of Day 2

16:30 – 17:00 — Deliberation

- Announcement of winners in the main categories
- Special prize for the mini-challenges

17:00 – 18:00 — Open retrospective

- What did you learn? What would you do differently?
- Open technical discussion among all teams and mentors
- Feedback collection to improve future editions

4. Evaluation Criteria

Important: the jury does not evaluate financial performance. A team whose agent loses money but reasons clearly, uses tools correctly, and handles errors gracefully will score higher than one that makes a profit by luck or by hardcoding decisions. The trading domain is the context — agentic behavior is the object of evaluation.

The jury evaluates three main dimensions. The maximum score is 100 points.

Criterion	Weight	What the jury evaluates
Demonstrable functionality	40 pt	The agent completes real end-to-end operations without crashing during the demo. It handles at least one tool error gracefully. The jury will interpret these criteria relative to the declared ambition level; Level 1 teams are not penalized for not executing orders.
Reasoning quality	35 pt	Decisions are well-motivated, traceable in the journal, and consistent with the data received. The model never invents prices or news. The agent knows when to stop: it does not act when data is insufficient or a tool is unavailable.
Originality of the use case	25 pt	The strategy or architecture shows something non-obvious: an unusual tool, a risk-management logic, an effective memory layer.

Special prize: Best Failure Recovery

Awarded to the team whose agent handles an unexpected error during the demo most intelligently (e.g. API down, anomalous price, rejected order). This prize does not count towards the main score but is announced separately.

Demo notes

- The demo must be live: no pre-recorded video.

- The agent must run autonomously for at least 5 consecutive minutes visible to the jury. The underlying system must be capable of sustaining autonomous operation for at least 5 minutes — this may be verified by the jury during Q&A.
- The trade journal must be displayable during the presentation.
- Context slides are optional.

5. Mini-Challenges

The mini-challenges are designed to keep energy high during coding sessions. They do not affect the final score, but each winner receives a symbolic recognition announced at the daily stand-up or the final awards ceremony.

Challenge	When	Goal	Prize
First Trade	Day 1 – lunch	First team to execute a real order on the simulated broker with documented reasoning	Title "First Mover"
Best Reasoning	Day 1 at 16:00	The clearest and most well-motivated reasoning step extracted from anyone's trade journal	Title "Best Analyst"
Best Recovery	Day 2 at 11:00	The team that demonstrates the most elegant handling of a tool error (injected by mentors)	Title "Anti-Fragile"
Best Journal	Day 2 at 13:00	The most readable and informative trade journal: structure, detail, usefulness	Title "Best Logger"

At 11:00 on Day 2, mentors silently inject a simulated error into each team's environment (e.g. the `get_price` tool returns a 503 error for 2 minutes). The team that best handles the fallback — without crashing and with a clear log — wins the challenge. Teams are notified only after the fact.

6. Preparation: What to Do the Day Before

Each GROUP, possibly each student, must complete the following steps before the hackathon begins. Teams that complete these steps in advance will be able to start coding immediately; otherwise, part of the first sprint will be spent on setup (typically 1-2 hours).

Mandatory setup — Mandatory per TEAM, recommended individually

1. Account and API keys

1. Create an account on Alpaca Markets (alpaca.markets) and activate Paper Trading.
2. Save the API Key and Secret Key in a local `.env` file (not in the repo).
3. Optional but recommended: create an account on Twelve Data or Polygon.io for additional market data.

2. Development environment

1. Python 3.13+ installed and working.
2. Create a virtual environment using “uv”.
3. Install the base dependencies: `alpaca-py`, `openai` (or `anthropic`), `python-dotenv`, `requests`.
4. Verify that the starter repo test code works: `python test_connection.py`

3. Starter repo

1. Fork the starter repo provided by the organizers on GitHub.
2. Read the full README and the ARCHITECTURE.md file.
3. Make at least one test call to the Alpaca API (get account balance) to verify your credentials.

4. Team formation

1. Agree with your teammates on who brings which laptop as the primary machine.
2. Decide in advance your ambition level (Level 1, 2, or 3 of the track).
3. Choose a team name and communicate it to the organizers.

Recommended readings (not mandatory)

- Official Alpaca Paper Trading API documentation (Orders and Positions sections).
- ReAct: Synergizing Reasoning and Acting in Language Models — original paper (arXiv 2022).
- Tool use / function calling in the documentation of your chosen LLM provider.
- An introductory article on RSI, MACD, and Moving Averages for those without a technical analysis background.

What to bring on hackathon day

- Laptop with the environment already configured and tested.
 - .env file with API keys already loaded (not printed, not on paper).
 - Access to the starter repo already forked and cloned locally.
 - Adapters and cables: availability on site is not guaranteed.
-

7. Recommended Tech Stack

Following this stack is not mandatory, but the starter repo is built around these tools.

- **Agent Framework:** LangGraph (recommended for loop control) or LangChain AgentExecutor.
- **Simulated Broker API:** Alpaca Paper Trading — simulated orders on real prices, free tier, REST and WebSocket API.
- **Market Data API:** Alpaca Market Data (included), or yfinance for historical data, or Twelve Data for real-time.
- **News API:** NewsAPI.org (free tier: 100 req/day) or Alpaca News Feed.
- **LLM Model:** OpenAI GPT-4o, Anthropic Claude Sonnet, or Gemini Flash — team's choice. As a fully offline fallback, Ollama with a local model (e.g. llama3 or mistral) requires no API key and works out of the box with LangChain.
- **Journaling Logging:** Local SQLite or structured JSON file — simple and inspectable.